

BroadcastSim v1.0 (Frozen)

Project Vision

BroadcastSim is a Digital Twin of a Broadcast Network.

Not a CRUD application.

Not a streaming application.

Not a monitoring dashboard.

It is a broadcast engineering simulator.

Final Tech Stack

Core Engine

- **Java 21**
- **Maven**

Web Application

- **Spring Boot**
 - **Thymeleaf**
 - **Bootstrap**
 - **Vanilla JavaScript**
 - **WebSocket**
 - **PostgreSQL**
-

Final Repository Structure

```
BroadcastSim/
|
├─ docs/
|   ├─ SRS.md
|   ├─ HLD.md
|   ├─ UML/
|   ├─ ERD/
|   ├─ API.md
|   └─ Images/
|
├─ broadcastsim-core/
|   ├─ src/main/java/com/broadcastsim/core
|   |
|   ├─ engine/
|   ├─ devices/
|   ├─ profiles/
|   ├─ properties/
|   ├─ rules/
|   ├─ signals/
|   ├─ ports/
|   ├─ connections/
|   ├─ events/
|   ├─ alarms/
|   ├─ registry/
|   ├─ timeline/
|   ├─ scenarios/
|   └─ util/
|
├─ broadcastsim-web/
|   ├─ Spring Boot
|   ├─ Controllers
|   ├─ WebSocket
|   ├─ Thymeleaf
|   ├─ Repository
|   ├─ Service
|   └─ Resources/
|
├─ README.md
├─ LICENSE
└─ .gitignore
```

Final Development Order

Sprint 1

- **Core enums**
 - **Property System**
 - **Device Framework**
-

Sprint 2

- **Simulation Engine**
 - **Clock**
 - **Registry**
 - **Event Queue**
-

Sprint 3

- **Signal Engine**
 - **Connections**
 - **Ports**
-

Sprint 4

- **Rule Engine**
-

Sprint 5

- **Devices**
- **Camera**
- **Router**
- **Encoder**
- **Decoder**
- **Media Server**
- **Viewer**

Sprint 6

- **Alarm Engine**
- **Timeline**

Sprint 7

- **Spring Boot Integration**

Sprint 8

- **UI**

Sprint 9

- **Save/Load Scenario**

Coding Principles

We will follow these throughout the project:

- **SOLID principles**
- **Clean Architecture (engine independent of web)**
- **Low coupling**
- **High cohesion**
- **Composition where appropriate**
- **Design patterns only where they solve a real problem**

Git Workflow

We'll create meaningful commits such as:

feat(core): add property system

feat(core): implement abstract device

feat(engine): add simulation scheduler

feat(signal): implement connection engine

feat(router): add routing logic

feat(web): integrate websocket dashboard

This gives the repository a professional history.

Documentation

We'll maintain:

- **README**
- **Architecture diagrams**
- **Class diagrams**
- **Sequence diagrams**
- **ER diagram**
- **API documentation**

The documentation evolves with the code instead of being written only at the end.

One Small Change (My Last Suggestion)

Instead of naming the engine:

SimulationEngine

I recommend calling it:

BroadcastEngine

Why?

Because over time it won't just "simulate." It will manage:

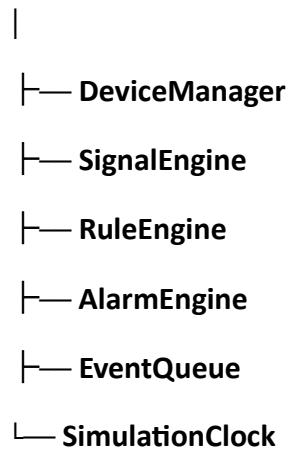
- **Device lifecycle**

- **Signal propagation**
- **Rules**
- **Events**
- **Alarms**
- **Time**
- **Scenarios**

It's effectively the engine of BroadcastSim.

Example:

BroadcastEngine



I think BroadcastEngine better reflects its role and sounds more like a reusable framework.